Ground-Truth Ego-Trajectory Reconstruction from Smartphone Dashcams

Architecture, estimation theory, findings, comparison with NVIDIA ViPE, and a plan toward a fused geometric video engine

TrajectoryReconstruction / TanitAD — technical report

8 November 2026

Abstract

We describe a pipeline that turns a consumer smartphone recording — camera video plus IMU, GNSS and orientation logs from the Sensor Logger app — into ground-truth ego trajectories for training end-to-end driving models, with a camera calibration that projects the trajectory into every frame. The report gives the full estimation theory: how video and sensors are put on one clock (container anchors refined by a three-band, three-axis gyro–image cross-correlation), how the phone's attitude in the vehicle is recovered (gravity, centripetal acceleration and GNSS sign checks), and how motion data are integrated into a trajectory: a six-state extended Kalman filter run forward over the whole session with the gyro and accelerometer as inputs and GNSS position, speed and course as measurements, followed by a Rauch–Tung–Striebel backward pass that gives every instant the information of the entire drive. We derive the process and measurement models, the robust update, the standstill constraints, the rigid transform from the phone's path to the rear axle, the steering read-out and the hold-out validation. We then report what we measured on five recordings from three phones, including a recording from a Galaxy S24 Ultra in an Audi A6 e-tron made with the ultra-wide lens, compare the approach with NVIDIA's ViPE (arXiv:2508.10934), and set out a plan that adopts ViPE's methods inside an estimator that keeps inertial and GNSS metric scale, explicit time synchronisation and a vehicle and road model. Numbers carry their evidence class: MEASURED (ours, with the artifact), INHERITED (another source, not re-verified), ESTIMATED, or HYPOTHESIS.

Contents

1 Problem and output **3**

2 Architecture **3**

3 Data model **4**

4 Quality gates **4**

5 Time synchronisation **4**

6 Phone-to-vehicle frame **5**

7 Longitudinal acceleration **5**

8 Trajectory estimation: forward filtering and backward smoothing **5**

8.1 Why one session-level estimate . . . 5
8.2 State and motion model . . . 6
8.3 Process noise . . . 6
8.4 Measurements . . . 6
8.5 Robust update . . . 7
8.6 Standstill and physical constraints . . . 7
8.7 Initialisation . . . 7
8.8 Backward pass: Rauch–Tung–Striebel smoother . . . 7
8.9 Ego windows and the vehicle reference point . . . 7
8.10 Steering read-out . . . 8
8.11 Validation . . . 8

9 Camera model and calibration **8**
9.1 Projection . . . 8
9.2 Ground plane and the $f \cdot h$ degeneracy . . . 8
9.3 Instruments . . . 8
9.4 Verification: prediction vs outcome . . . 9

10 Findings **9**
10.1 Defects in the earlier implementation . . . 9
10.2 Trajectory accuracy . . . 9
10.3 Calibration failures and their causes . . . 9
10.4 Electronic image stabilisation moves the camera . . . 9
10.5 Galaxy S24 Ultra in an Audi A6 e-tron (2026-11-03) . . . 10
10.6 Corrected S24 render . . . 10

11 NVIDIA ViPE **11**
11.1 Pipeline (paper §3.1) . . . 11
11.2 Formulation (paper Eqs. 1–5) . . . 11
11.3 Dense depth alignment (paper Eq. 6) . . . 11
11.4 Reported results (INHERITED) . . . 11
11.5 Comparison . . . 12

12 Plan: a fused geometric video engine **12**
12.1 Target formulation . . . 12
12.2 What is adopted from ViPE, and how it is developed further . . . 13
12.3 Work packages . . . 13

13 Recording recommendations **13**

14 Limitations **14**

1 Problem and output

For every video frame the pipeline outputs the vehicle's past and future trajectory in an ego frame over $t \in [-3, +5]$ s: positions, heading, speed and positional uncertainty, together with the steering-wheel angle implied by the motion, a bird's-eye view and the future path drawn into the image. The ego frame is FLU (ISO 8855: x forward, y left, z up) with its origin at the rear axle at the reference instant.

The inputs are deliberately cheap: a phone in a windscreen holder recording with Sensor Logger. There is no vehicle CAN, no calibration target and no control over the camera mode the app uses. Two consequences drive the design: the pipeline must *refuse* recordings that cannot yield trustworthy ground truth, and it must recover all geometry — time offset, mount attitude, intrinsics — from the drive itself.

2 Architecture

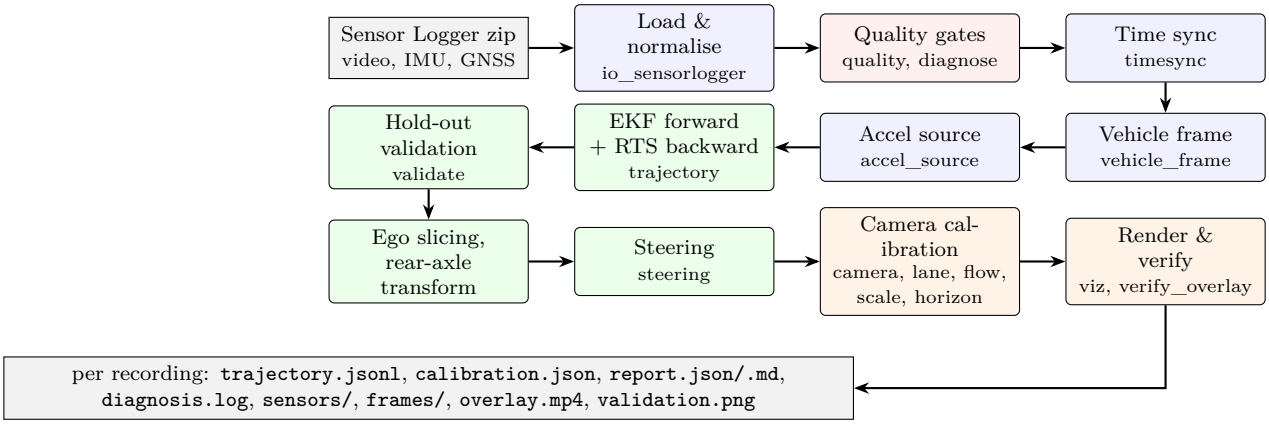


Figure 1: Data flow. Blue: data preparation; red: refusal gates; green: trajectory; orange: camera and rendering. The sync result feeds both the trajectory window and every frame's time stamp.

Driver. `pipeline.py` consumes a folder of Sensor Logger `.zip` archives directly. A registry keyed on each archive's input fingerprint makes runs resumable and idempotent: completed work is skipped, failures are retried, changed inputs are reprocessed, and an interruption costs at most one recording. Work happens in a local scratch directory and is published at the end (the source folder is a Google Drive mount that has dropped out mid-run). Each recording is *diagnosed before any solving*; a rejected recording keeps its written analysis.

module	role
`io_sensorlogger`	loads all streams onto one timebase; detects platform, accelerometer units, iOS -1 sentinels
`quality`, `diagnose`	frozen GNSS, rates, gaps, motion, accelerometer usability; OK/DEGRADED/REJECT with evidence
`timesync`	container PTS, coarse anchors, image angular rate, three-band gyro cross-correlation
`vehicle_frame`	one phone→vehicle rotation per session
`accel_source`	tiered choice of longitudinal acceleration, with automatic quality scoring
`trajectory`	session EKF + RTS smoother; ego slicing; rigid transform to the rear axle
`validate`	k-fold GNSS hold-out and internal consistency
`steering`	curvature, bicycle model with understeer, rate limits, uncertainty
`camera`, `lane_calib`, `scale_calib`, `flow_calib`, `horizon_calib`, `plane_calib`, `vp_calib`, `ground_calib`	intrinsics and mount from the video (§9)
`attitude_track`	per-frame yaw/horizon for recordings with electronic stabilisation
`viz`, `tools/verify_overlay`	ribbon and bird's-eye rendering; lane-based verification

Table 1: Modules of the `trajlib` library.

3 Data model

All streams are expressed in session time t (`seconds_elapsed`), seconds since the recording epoch T_0 written in `Metadata.csv`; absolute time is $T_0 + t$. Platform conventions are detected from the data: Android's gravity vector points up and iOS's down; iOS exports the raw accelerometer in g (recognised by a mean norm of 1 g), Android in m/s^2; iOS writes -1 for unavailable speed, course and accuracy, which would otherwise poison the filter (one such value made a whole iOS state NaN). GNSS fixes are converted to a local ENU tangent plane at the first fix (WGS-84 $\rightarrow$ ECEF $\rightarrow$ ENU). Headings use the mathematical convention ψ (counter-clockwise from East); a GNSS bearing β (clockwise from North) maps to $\psi = \pi/2 - \beta$.

4 Quality gates

Before any solving: GNSS frozen fraction (byte-identical consecutive fixes — one recording's 78 fixes were all one cached fix), fix count and gaps, receiver accuracy, IMU rates, motion (distance travelled, moving fraction), overlap of video and sensor data, and the usable accelerometer tier. A REJECT stops the recording with its report; DEGRADED proceeds with the reason recorded. The design rule is that a silently wrong trajectory is worse than none, because it poisons a training set unnoticed.

5 Time synchronisation

Every frame needs a session time $t_k = t_0 + p_k$, where p_k is the frame's presentation timestamp (PTS) read from the container and t_0 the session time of the first frame. Using PTS rather than k/fps matters: at a measured 29.9436 fps against a nominal 30, index arithmetic drifts 1.7 s over 78 s.

Coarse anchors. Three candidates for t_0 are formed: the epoch encoded in the video file name (*container*), and the MP4 `creation_time` T_c read either as the start ($T_c - T_0$) or as the end ($T_c - T_0 - D$, with duration D). Android writes T_c when the muxer finalises the file (the end), iOS at the start; the first candidate inside a physically possible window (-3 to $+15$ s) is used.

Image angular rate. Corners are tracked with pyramidal Lucas–Kanade between consecutive frames in a band that excludes sky and the static bonnet/dashboard (RANSAC would otherwise lock onto the static majority). A partial-affine (similarity) transform is fitted; with a nominal focal length f, the camera's incremental rotations are $\Delta\theta_{\text{yaw}} \approx t_x/f$, $\Delta\theta_{\text{pitch}} \approx t_y/f$ and $\Delta\theta_{\text{roll}}$ = the in-plane angle, giving $\boldsymbol{\omega}_{\text{cam}}(t) \in \mathbb{R}^3$. Forward translation leaks into t_x, so the amplitude is unreliable; only the timing is used.

Three-axis lag and axis map. Both signals are resampled on a common grid ($\Delta = 20$ ms) and band-passed with a zero-phase Butterworth filter (removes bias drift and high-frequency noise). For each candidate lag $\tau \in [-3, 3]$ s the gyro triad $\mathbf{B}(\tau) \in \mathbb{R}^{n\times 3}$, sampled at $t + \tau$, is mapped onto the camera triad $\mathbf{A} \in \mathbb{R}^{n\times 3}$ by a free 3×3 matrix:

$$\mathbf{M}(\tau) = \arg\min_{\mathbf{M}} \|\mathbf{A} - \mathbf{B}(\tau)\mathbf{M}\|_F^2, \qquad s(\tau) = 1 - \frac{\|\mathbf{A} - \mathbf{B}(\tau)\mathbf{M}(\tau)\|_F^2}{\|\mathbf{A}\|_F^2}. \tag{1}$$

$\mathbf{M}$ is a general matrix rather than a rotation on purpose: it absorbs the unknown phone$\rightarrow$camera axis mapping *and* the per-axis gain error of the image rates. The score s is the fraction of camera-signal variance explained; $\hat{\tau} = \arg\max s$, refined to sub-sample precision by a parabola through the peak: $\hat{\tau} \mathrel{+}= \Delta\, \frac{1}{2}(s_- - s_+)/(s_- - 2s_0 + s_+)$.

Acceptance. The fit is repeated in three bands, $[0.02, 1]$, $[0.05, 2]$ and $[0.10, 3]$ Hz. A true lag is band-independent; a spurious peak moves. The refinement is accepted only if the three lags agree within 100 ms and the peak score exceeds 0.10; then $t_0 \leftarrow t_0 + \text{median}(\hat{\tau})$, otherwise the coarse anchor is kept and the rejection is recorded.

recording	old pipeline (first GNSS fix)	this method	agreement
2025-08-14	0.606 s	0.2134 s	3 bands within 1 ms, score 0.59
2025-08-11	1.037 s	0.7327 s	3 bands within 96 ms, score 0.89
Pacific Coast Hwy (iPhone 14)	—	0.1068 s	within 7 ms, score 0.93
2026-11-03 1000N (S24 Ultra)	—	0.47 s	within 35 ms, score 0.82 (MEASURED)

Table 2: Video start in session time (first three rows INHERITED from AUDIT.md).

6 Phone-to-vehicle frame

The phone sits at an unknown attitude, fixed for the session. One rotation $\mathbf{R}_{vp}$ (phone to vehicle) is estimated from all the data, with its three axes found independently:

- **Up $\hat{\boldsymbol{u}}$**: the mean of the gravity stream, $\hat{\boldsymbol{u}} = \overline{\boldsymbol{g}}/\|\overline{\boldsymbol{g}}\|$. Its sign is *measured*: the rotation rate about $\hat{\boldsymbol{u}}$, $\omega_u = \boldsymbol{\omega} \cdot \hat{\boldsymbol{u}}$, is correlated with the GNSS heading rate $\dot{\psi}_{\text{GNSS}}$ over moving fixes; a negative correlation flips $\hat{\boldsymbol{u}}$. (Assuming Android's convention inverted every turn on iOS: $r = -0.43$ on Rose Ave.) Without a gravity stream, $\hat{\boldsymbol{u}}$ is the principal axis of the gyro signal, since a road vehicle rotates almost only in yaw.
- **Left $\hat{\boldsymbol{l}}$**: from the centripetal relation $a_{\text{lat}} = v\,\omega_{\text{up}}$. With the horizontal linear acceleration $\boldsymbol{a}_h = \boldsymbol{a} - (\boldsymbol{a} \cdot \hat{\boldsymbol{u}})\hat{\boldsymbol{u}}$ and a speed gate $w(t)$,

$$\hat{\boldsymbol{l}} \propto \Big(\textstyle\sum_t w_t\, \omega_{\text{up},t}\, \boldsymbol{a}_{h,t} \Big)_{\perp \hat{\boldsymbol{u}}}, \tag{2}$$

  the horizontal direction whose acceleration best correlates with the yaw rate. The normalised correlation is reported as `lateral_score`.
- **Forward $\hat{\boldsymbol{f}} = \hat{\boldsymbol{l}} \times \hat{\boldsymbol{u}}$** (right-handed FLU). Its sign is fixed by the camera boresight (the rear camera looks along $-z_{\text{phone}}$, which has a positive forward component), and checked against the GNSS longitudinal acceleration when GNSS is healthy.

Then $\mathbf{R}_{vp} = [\hat{\boldsymbol{f}}\ \hat{\boldsymbol{l}}\ \hat{\boldsymbol{u}}]^\top$ and any phone-frame vector maps as $\boldsymbol{v}_{\text{veh}} = \mathbf{R}_{vp}\boldsymbol{v}_{\text{phone}}$. The vehicle yaw rate used by the filter is $\omega = (\mathbf{R}_{vp}\boldsymbol{\omega}_{\text{gyro}})_z$.

7 Longitudinal acceleration

Android's `Accelerometer` stream is *linear* acceleration: specific force minus a continuously re-estimated gravity whose time constant is about a second, so sustained acceleration leaks into the gravity estimate and is subtracted away. Measured on 2025-08-11 its forward component integrates to -0.8 m/s while the car gained $+7.8$ m/s (correlation with GNSS dv/dt: -0.06). The source is therefore chosen in tiers:

1. raw specific force $\boldsymbol{a}^{\text{tot}}$ with gravity removed through the gyro-led attitude quaternion, $\boldsymbol{a} = \mathbf{R}(q)^\top \boldsymbol{a}^{\text{tot}} - \boldsymbol{g}$ in the world frame;
2. $\boldsymbol{a}^{\text{tot}}$ minus the `Gravity` stream;
3. the linear stream;
4. none: $a_f \equiv 0$ and the process noise is raised so speed is a random walk driven by GNSS.

Each candidate's forward component $a_f = (\mathbf{R}_{vp}\boldsymbol{a})_x$ is scored by its agreement with GNSS-derived acceleration; a source below the quality threshold is disabled rather than allowed to inject noise. Unknown quality (e.g. NaN from a single missing speed) is treated as unusable, not as trusted.

8 Trajectory estimation: forward filtering and backward smoothing

8.1 Why one session-level estimate

The earlier implementation solved a fresh filter inside each frame's window $[t_k, t_k + 5\,\text{s}]$: no past, neighbouring frames 33 ms apart disagreeing, $\approx$5 GNSS fixes per solve and 2 GB of duplicated sensor data per 78 s. We instead estimate the *whole session once* — a forward pass that is causal, then a backward pass that propagates future information — and slice every frame's window from that single smoothed solution. Every frame then carries the full past and future, and adjacent frames are consistent by construction.

8.2 State and motion model

On a uniform grid with $\Delta t = 10\,\text{ms}$ the state is

$$\boldsymbol{x} = [\,E,\ N,\ v,\ \psi,\ b_\omega,\ b_a\,]^\top \in \mathbb{R}^6, \tag{3}$$

position in the ENU plane, forward speed, heading, gyro yaw-rate bias and longitudinal accelerometer bias. The vehicle is non-holonomic (it moves along its heading). The control inputs are the vehicle-frame yaw rate ω_k and longitudinal acceleration $a_{f,k}$. With $a = a_f - b_a$, $\tilde{\omega} = \omega - b_\omega$, $c = \cos\psi$, $s = \sin\psi$, the discrete propagation is

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k) = \begin{bmatrix} E + vc\,\Delta t + \frac{1}{2}ac\,\Delta t^2 \\ N + vs\,\Delta t + \frac{1}{2}as\,\Delta t^2 \\ v + a\,\Delta t \\ \psi + \tilde{\omega}\,\Delta t \\ b_\omega \\ b_a \end{bmatrix}. \tag{4}$$

Its Jacobian $\mathbf{F}_k = \partial f / \partial \boldsymbol{x}$ is the identity plus

$$\begin{aligned} &\mathbf{F}_{Ev} = c\,\Delta t, \quad \mathbf{F}_{E\psi} = -\big(v\Delta t + \tfrac{1}{2}a\Delta t^2\big)s, \quad \mathbf{F}_{Eb_a} = -\tfrac{1}{2}c\,\Delta t^2, \\ &\mathbf{F}_{Nv} = s\,\Delta t, \quad \mathbf{F}_{N\psi} = \big(v\Delta t + \tfrac{1}{2}a\Delta t^2\big)c, \quad \mathbf{F}_{Nb_a} = -\tfrac{1}{2}s\,\Delta t^2, \\ &\mathbf{F}_{vb_a} = -\Delta t, \quad \mathbf{F}_{\psi b_\omega} = -\Delta t. \end{aligned} \tag{5}$$

Gyro pre-filter. A road vehicle's yaw dynamics lie below ≈1.5 Hz (98.2 % of the yaw-rate power is under 0.5 Hz on 2025-08-11). The rest is body roll and mount flex; integrated, it puts a $\approx 0.3°$ wobble into heading (0.25 m lateral at 45 m). Because ω is the propagation *input*, no measurement-side tuning can remove it; it is low-passed at 1.5 Hz with a zero-phase (forward–backward) second-order Butterworth filter, so no lag is introduced.

8.3 Process noise

Along-track uncertainty follows the continuous white-noise-acceleration model, discretised exactly, which is positive semi-definite by construction (a hand-built cross term must satisfy $Q_{pp}Q_{vv} \geq Q_{pv}^2$ or the covariance goes indefinite and the RTS pass degrades):

$$\mathbf{Q}_{pp} = \sigma_a^2 \frac{\Delta t^3}{3}\,\boldsymbol{e}\boldsymbol{e}^\top + \sigma_{\text{lat}}^2 \Delta t\,\boldsymbol{e}_\perp \boldsymbol{e}_\perp^\top, \quad \mathbf{Q}_{pv} = \sigma_a^2 \frac{\Delta t^2}{2}\,\boldsymbol{e}, \quad \mathbf{Q}_{vv} = \sigma_a^2 \Delta t, \tag{6}$$

$$\mathbf{Q}_{\psi\psi} = \sigma_\omega^2 \Delta t, \quad \mathbf{Q}_{b_\omega b_\omega} = \sigma_{b\omega}^2 \Delta t, \quad \mathbf{Q}_{b_a b_a} = \sigma_{ba}^2 \Delta t, \tag{7}$$

with $\boldsymbol{e} = (c, s)$ the heading direction and $\boldsymbol{e}_\perp = (-s, c)$. The small lateral term ($\sigma_{\text{lat}} = 0.05$) models tyre slip and mount flex and keeps $\mathbf{P}$ non-singular across track. Defaults: $\sigma_a = 0.6$ (0.8 when no accelerometer is usable, tuned on hold-out subject to a physical cap on p99 acceleration), $\sigma_\omega = 0.02$, $\sigma_{b\omega} = 2 \cdot 10^{-4}$, $\sigma_{ba} = 5 \cdot 10^{-3}$. While the vehicle is stationary all noise terms are scaled by 0.05: a stopped car cannot accelerate, slip or — the one that matters — rotate. At $v = 0$ heading is unobservable (position carries no heading information and the GNSS course is gated off), and at driving-level noise it random-walked $\approx 5°$ through an 8 s stop.

8.4 Measurements

GNSS fixes are snapped to the nearest grid step. All updates are scalar with $\mathbf{H}$ a unit row:

- position z_E, z_N with variance $\sigma^2 = h_{\text{acc}}^2$ (the receiver's reported horizontal accuracy; inflating it measurably *worsened* hold-out position, 2.23 → 2.87 m at 1.5×);
- speed z_v with $\sigma^2 = s_{\text{acc}}^2$ (default 0.6 m/s);
- course $z_\psi = \pi/2 - \beta$ only when $v \geq 2.5$ m/s, with σ from `bearingAccuracy` clipped to $[2°, 60°]$ (default 12° when the device reports 0, which means "unknown", not "perfect"); the innovation is wrapped to $(-\pi, \pi]$.

8.5 Robust update

For innovation $\nu = z - \mathbf{H}\boldsymbol{x}^-$ and $S = \mathbf{H}\mathbf{P}^-\mathbf{H}^\top + R$, the normalised innovation squared is NIS $= \nu^2/S$. A hard χ^2 gate that discards outliers is a trap here: after a stop the zero-velocity update drives P_{vv} to almost nothing, the first GNSS speed after pulling away looks like an outlier, every update is rejected and the filter never recovers (observed: speed pinned near zero for 25 s). We therefore *down-weight* instead of rejecting (Huber-style):

$$R' = R \cdot \max\big(1, \mathrm{NIS}/g\big), \qquad g = 16 \ (\approx 4\sigma), \tag{8}$$

$$\boldsymbol{K} = \mathbf{P}^-\mathbf{H}^\top/S', \quad \boldsymbol{x}^+ = \boldsymbol{x}^- + \boldsymbol{K}\nu, \quad \mathbf{P}^+ = (\mathbf{I} - \boldsymbol{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \boldsymbol{K}\mathbf{H})^\top + \boldsymbol{K}R'\boldsymbol{K}^\top, \tag{9}$$

the Joseph form, which stays symmetric positive semi-definite in finite precision.

8.6 Standstill and physical constraints

- **Zero-velocity update (ZUPT).** A stationary receiver does not report 0.00 m/s (Doppler noise keeps it at a few tenths), so the threshold is 0.7 m/s, and a fix is called stationary only if it and both neighbours are below it (a single isolated 0.0 at walking pace punched a false stop-and-launch into the speed). Between two stationary fixes less than 3 s apart, the pseudo-measurement $z_v = 0$, $\sigma = 0.25$ m/s is applied at *every* propagation step, not only at fix instants.
- **Non-negative speed.** A forward dashcam never reverses, and a negative excursion flips the heading interpretation of every exported window; if $v < 0$ a pseudo-measurement $z_v = 0$, $\sigma = 0.05$ is applied, so the covariance stays consistent rather than the state being clamped.

8.7 Initialisation

From the first fix with a finite value: position from the first fix, v_0 from the first finite speed, ψ_0 from the first finite course; $\mathbf{P}_0 = \mathrm{diag}(25, 25, 4, (\pi/2)^2, 10^{-3}, 0.5)$.

8.8 Backward pass: Rauch–Tung–Striebel smoother

The forward pass stores the predicted $(\boldsymbol{x}_k^-, \mathbf{P}_k^-)$, filtered $(\boldsymbol{x}_k^+, \mathbf{P}_k^+)$ and Jacobians $\mathbf{F}_k$. Starting from $\boldsymbol{x}_N^s = \boldsymbol{x}_N^+$, for $k = N-1, \ldots, 0$:

$$\mathbf{C}_k = \mathbf{P}_k^+\mathbf{F}_{k+1}^\top(\mathbf{P}_{k+1}^-)^{-1}, \qquad \boldsymbol{x}_k^s = \boldsymbol{x}_k^+ + \mathbf{C}_k\big(\boldsymbol{x}_{k+1}^s - \boldsymbol{x}_{k+1}^-\big), \qquad \mathbf{P}_k^s = \mathbf{P}_k^+ + \mathbf{C}_k\big(\mathbf{P}_{k+1}^s - \mathbf{P}_{k+1}^-\big)\mathbf{C}_k^\top, \tag{10}$$

with the heading component of $\boldsymbol{x}_{k+1}^s - \boldsymbol{x}_{k+1}^-$ wrapped to $(-\pi, \pi]$ before use, and a pseudo-inverse if $\mathbf{P}^-$ is ill-conditioned. The smoothed estimate is the minimum-variance estimate given *all* measurements of the session; this is what turns a causal navigation estimate into ground truth, and it is why GNSS outages are bridged from both sides.

8.9 Ego windows and the vehicle reference point

At a frame time t_r the smoothed states are interpolated on $t_r + \tau$, $\tau \in [-3, 5]$ s ($\Delta\tau = 0.1$ s), and rotated into the ego frame:

$$\begin{bmatrix} x(\tau) \\ y(\tau) \end{bmatrix} = \begin{bmatrix} \cos\psi_r & \sin\psi_r \\ -\sin\psi_r & \cos\psi_r \end{bmatrix} \begin{bmatrix} E(t_r{+}\tau) - E_r \\ N(t_r{+}\tau) - N_r \end{bmatrix}, \qquad \mathrm{yaw}(\tau) = \psi(t_r{+}\tau) - \psi_r. \tag{11}$$

GNSS locates the phone, mounted L ahead of and W left of the rear axle. Rigid-body points trace different curves, so the rear axle's path is $\boldsymbol{p}_{\mathrm{ref}} = \boldsymbol{p}_{\mathrm{phone}} - \mathbf{R}(\psi)(L, W)^\top$; in ego coordinates

$$x_{\mathrm{ref}} = x + L(1 - \cos\psi) + W\sin\psi, \qquad y_{\mathrm{ref}} = y - L\sin\psi + W(1 - \cos\psi), \tag{12}$$

and speed is recomputed along the transformed path. A constant shift $y - W$, used before, leaves an error $-L\sin\psi + W(1 - \cos\psi)$: zero on straight road, 2.57 m at the 5 s horizon on Rose Ave with $L = 2.4$ m. When the vehicle is genuinely stopped ($|v| < 0.15$ m/s) the future window is shortened to 1 s and flagged, so long stops do not flood the data set with identical zero trajectories.

8.10 Steering read-out

Path curvature is regularised at low speed, $\kappa = \omega v/(v^2 + v_\epsilon^2)$ with $v_\epsilon = 1.5\,\mathrm{m/s}$, and the road-wheel angle follows a kinematic bicycle model with a linear understeer term,

$$\delta = \arctan(L_{\mathrm{wb}}\,\kappa) + K_{\mathrm{us}}\,a_{\mathrm{lat}}, \qquad \theta_{\mathrm{wheel}} = r_s\,\delta, \tag{13}$$

(Audi A6 e-tron: $L_{\mathrm{wb}} = 2.946\,\mathrm{m}$, $r_s = 15.9$). The uncertainty propagates from yaw rate and speed; speed dominates (1.3 m/s RMS is 12 % at 11 m/s but 40 % at 3 m/s), and samples below 1.5 m/s are marked invalid. A rate limit (180 °/s) applies to the read-out only; rewriting the path itself with rate-limited curvature displaced the 5 s window by up to 2.48 m in a turn to remove 5 mm of ripple, so it is off by default.

8.11 Validation

Hold-out: for $i = 0..k-1$ ($k = 5$), every k-th GNSS fix starting at i is removed, the whole session re-estimated, and the estimate scored at the removed fixes; at 1 Hz this bridges ≈5 s outages, so it measures prediction, not interpolation. **Internal consistency:** $\|d\boldsymbol{p}/dt\|$ vs the speed state, course vs heading, minimum speed, positional 1σ, p99 longitudinal acceleration and yaw rate. Tuning decisions (e.g. h_{acc} inflation, σ_a without IMU) were made by minimising hold-out error subject to physical plausibility, not by eye.

9 Camera model and calibration

9.1 Projection

Pinhole intrinsics (f_x, f_y, c_x, c_y); the camera is at $\boldsymbol{t}_v = (L, W, h)$ in the vehicle frame with mount rotation $\mathbf{R}_{cv} = \mathbf{R}(\mathrm{yaw}, \mathrm{pitch}, \mathrm{roll})\,\mathbf{R}_0$ ($\mathbf{R}_0$ maps FLU to OpenCV). A vehicle-frame point projects as $\boldsymbol{p}_c = \mathbf{R}_{cv}(\boldsymbol{p}_v - \boldsymbol{t}_v)$, $u = f_x p_{c,x}/p_{c,z} + c_x$, $v = f_y p_{c,y}/p_{c,z} + c_y$, valid only for $p_{c,z} > 0.35\,\mathrm{m}$. The horizon row is $v_h = f_y d_y/d_z + c_y$ with $\boldsymbol{d} = \mathbf{R}_{cv}(1,0,0)^\top$.

9.2 Ground plane and the $f \cdot h$ degeneracy

For a flat road and a ground point at range X and lateral offset Y from the camera,

$$v = v_h + \frac{f\,h}{X}, \qquad u = c_x + f\,\frac{Y}{X} \quad\Longrightarrow\quad Y = (u - c_x)\,\frac{h}{v - v_h}. \tag{14}$$

Laterally only h appears; longitudinally only fh. No check that uses only the ground plane can separate f from h: a wrong focal length is absorbed into a wrong height and passes every self-consistency test. The horizon is a separate, high-leverage parameter: an error scales the drawn width by $(v - v_h^{\mathrm{true}})/(v - v_h^{\mathrm{used}})$ (11 % at 30 m for 9 px).

9.3 Instruments

- **Focal length from the gyro**: regression of image yaw rate on gyro yaw rate.
- **Focus of expansion** (yaw, pitch): flow lines on straight driving meet at the FOE; the rotation gate reads the gyro (the image rate leaks translation at speed: at 70–84 km/h the image gate passed 0.4 % of frames, the gyro gate gave 425,093 vectors at 86 % inliers).
- **Lane vanishing point** (yaw): both boundaries fitted as lines, VP column u^* gives yaw = $\arctan((u^* - c_x)/f)$ on straight frames; half-split agreement as acceptance.
- **Lateral offset** from the ego-lane centre, assuming lane-centred driving on average.
- fh **from motion** (`scale_calib`, `flow_calib`): a static ground point satisfies $1/(v - v_h) = (X_0 - s)/(fh)$ as the car travels s; `flow_calib` also solves for v_h.
- **Horizon from lane-width constancy** (`horizon_calib`): a lane of width W subtends $w_{\mathrm{px}}(v) = (W/h)(v - v_h)$, a straight line in v whose x-intercept is v_h; no lane width needed.
- **Attitude track** for stabilised video: yaw(t) and $v_h(t)$ applied per frame when drawing.

9.4 Verification: prediction vs outcome

Because the future is known, the drawing can be scored without assuming the driver is centred. Let $\phi_F(X)$ be the ribbon centre's lane fraction at range X in frame F (0 at the lane centre). Find the later frame F' in which the car has travelled $X - X_0$ along the path; then the same patch of road is at range X_0 (near field) in F', and

$$e(X) = \big(\phi_F(X) - \phi_{F'}(X_0)\big) \cdot W_{\text{lane}} \tag{15}$$

is the drawing error at range X. It assumes only that the paint stays put. `tools/verify_overlay.py` also reports the angle between the drawn path and the lane (both boundaries fitted as whole lines) and the offset from the lane centre at 10 and 25 m, on straight frames with a plausible lane width only.

10 Findings

10.1 Defects in the earlier implementation

id	finding (INHERITED from AUDIT.md)
F1	`creation_time` is the *end* of an Android recording, not the start.
F2	Sync anchored on the first GNSS fix: +0.30 s residual on 2025-08-11 (3.3 m, 9 frames at 11 m/s).
F3	Per-frame 5 s filter windows: no past, inconsistent neighbours, 2 GB duplicated per 78 s.
F4	No quality gate: a frozen cached fix would have produced a smooth, wrong line.
F5–F7	`bearingAccuracy`=0 read as perfect; single-axis IMU alignment per window; IMU mean forced to GNSS mean per window, measurement used as its own control input.
F10	The Galaxy accelerometer carries no usable longitudinal signal (correlation 0.06).
F11	Gravity sign differs between Android and iOS; the frame was inverted on iOS.
F12	Unknown quality treated as trusted; one NaN disabled two safeguards (hold-out 76.8 m → 2.41 m once fixed).
F13	FOE yaw biased by curves (0.98°); the lane VP fixes it.
F14	Focal length unobservable from the ground plane; five methods, five answers.
F15	The trajectory was the phone's path, not the vehicle's (2.57 m in turns).

10.2 Trajectory accuracy

recording	phone	position RMS	speed RMS	heading	class
2025-08-11	Galaxy SM-G990B	2.23 m	1.27 m/s	0.84° RMS	INHERITED (AUDIT.md)
2026-08-08 14-19-54	SM-G990B	0.70 m	0.08 m/s	0.53°	MEASURED (TanitAD study)
2026-11-03 1000N	S24 Ultra	1.36 m	0.27 m/s	p95 1.53°	MEASURED (this report)

Table 3: Hold-out accuracy (every fifth fix withheld). Median receiver accuracy is 3.8 m on all three.

10.3 Calibration failures and their causes

Each was logged with its root-cause class (`docs/CALIBRATION_RETRACTIONS.md`): a yaw gate that judged a measured −7.01° against a FOE that had failed (4.9 m error at 40 m); a rotation gate fed by a timing instrument; instruments that measured the wrong object (a roadside bank, a tar seam, foliage, shadowed asphalt, the dashed line wandering); metrics blind to the horizon, which was 9 px out; frame mismatches (a camera range applied in the rear-axle frame painted the ribbon on the bonnet); and a pooled statistic hiding a time-dependent failure.

10.4 Electronic image stabilisation moves the camera

On 2026-08-08 (VW Caddy, EIS on) the lane vanishing point walked from column ≈751 to ≈830 over 50 s — about 3° of yaw, correlated with time ($r = -0.41$), not steering ($r = +0.09$). No constant yaw can fit. A per-frame attitude track (cowl-edge registration fused with a low-passed prediction-vs-outcome correction, fitted on half the frames, tested on the other half) brought the flagged frame from −1.75° to −0.16° path-vs-lane, and the held-out prediction-vs-outcome spread at 30 m from 0.37 m to 0.19 m (MEASURED). Over the whole render, median path-vs-lane angle −0.61°, p10/p90 −1.38/ + 0.76° over 97 straight frames.

10.5 Galaxy S24 Ultra in an Audi A6 e-tron (2026-11-03)

11.2 min, 8.66 km, up to 26.7 m/s; phone in front of the rear-view mirror; stabilisation off. Trajectory, sync and vehicle frame worked first time. The camera failed for four reasons, each an unstated assumption:

1. **Resolution**: 3840×2160; stages tuned in 1080p pixels (e.g. horizon bracket 380–580 px).
2. **Timestamps through a re-encode**: ffmpeg 6.1 with `-fps_mode passthrough` still snapped frame times to an exact 30.000 fps grid (phone: ≈30.002 fps), moving 19,466 of 20,176 frames by more than 1 ms, up to 16.7 ms (0.4 m at 25 m/s); `-enc_time_base:v demux` keeps them exact (0.0 ms).
3. **Lens**: the *ultra-wide* camera. Far features shift by $f\Delta\psi$ in a bend: $f \approx 695$ px (HFOV ≈ 108°). Dash ends moving a known distance give $fh = 913$ px m (IQR 816–972, 1,003 pairs), so $h \approx 1.30$ m. The gyro regression gave 1857 px and the 55–85° plausibility gate would have rejected the true value; whole-image flow gave $fh = 1098$, biased by static interior pixels.
4. **Visible road band**: the dashboard fills the lower third; the lane search band was mostly dashboard (yaw +1.6°, lateral +0.8 m), and the first render put the ribbon on the verge.

On the visible band the lane lines meet at $(842, 544)$ on straight frames: horizon 544, yaw $\arctan((842 - 960)/700) = -9.6°$. Corrected parameters: $f = 700$ px, yaw −9.6°, horizon 544, $h = 1.30$ m, lateral 0.

10.6 Corrected S24 render

Rendered for $t = 150$–210 s (1,800 frames, 2,052 s end to end on 4 CPU cores including calibration) with the corrected parameters; calibration and the trajectory still used the full 11-minute recording. Fig. 2 shows a frame in the right-hand bend at $t = 180$ s, where the first render had put the ribbon on the verge.



Figure 2: S24 Ultra / A6 e-tron, $t = 180.06$ s, 23.3 m/s, yaw rate −7.3°/s. The future path (green) stays in the ego lane through the bend; the bird's-eye panel shows the same right turn that the GNSS bearing confirms (118° → 151° over 5 s).

metric (`tools/verify_overlay.py`, every 3rd frame)	value
frames sampled / straight with both lines / curved or slow	602 / 29 / 535
path vs lane angle, median (p10, p90)	−0.14° (−0.96°, +0.84°)
offset from lane centre at 10 m, median	−0.06 lane widths
offset from lane centre at 25 m, median	−0.05 lane widths
ribbon centre inside the lane to 30 m	29 / 29 frames
ribbon edges (1.923 m) inside the lane to 30 m	26 / 29 frames
prediction vs outcome at 20 m, median (p10, p90)	+0.01 m (−0.87, +0.10), $n = 27$
prediction vs outcome at 30 m, median (p10, p90)	+0.01 m (−0.81, +0.14), $n = 24$

Table 4: Corrected render, MEASURED (`verify_fix.json`). The window is mostly bends, so only 29 frames qualify as straight with two visible lines; the numbers are encouraging but a small sample, and the −0.8 m tail of prediction vs outcome is not yet explained. The full-length render and a check over all 11 minutes are the next step.

11 NVIDIA ViPE

ViPE (Huang et al., "ViPE: Video Pose Engine for 3D Geometric Perception", NVIDIA, arXiv:2508.10934v1, 12 Aug 2025; code https://github.com/nv-tlabs/vipe) estimates camera intrinsics, poses and dense near-metric depth from unconstrained video. The following is from the paper (INHERITED: not reproduced here).

11.1 Pipeline (paper §3.1)

DROID-SLAM-style keyframe SLAM: (1) intrinsics initialised by GeoCalib on 4 sampled frames; (2) keyframe selection when predicted motion (weighted dense flow plus sparse tracks) exceeds a threshold; (3) frontend BA in a sliding window over recent keyframes, edges by temporal proximity or co-visibility, Gauss–Newton; (4) backend BA over all keyframes with intrinsics unlocked, run at 8, 16, 64 keyframes and at the end; (5) pose infilling of non-keyframes from their two nearest keyframes; (6) dense depth per frame.

11.2 Formulation (paper Eqs. 1–5)

Unknowns: poses $\{\mathbf{T}_i \in \mathrm{SE}(3)\}$, intrinsics $\boldsymbol{k}$, low-resolution keyframe depth $\mathbf{D}_i \in \mathbb{R}^{H/8 \times W/8}$:

$$e_{\mathrm{ViPE}} = \sum_{(i,j)\in\mathcal{E}} e_{\mathrm{dense}}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{D}_i, \boldsymbol{k}) + \sum_{(i,j)\in\mathcal{E}} e_{\mathrm{sparse}}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{D}_i, \boldsymbol{k}) + \alpha \sum_{i\in\mathcal{V}} e_{\mathrm{depth}}(\mathbf{D}_i), \tag{16}$$

$$e_{\mathrm{dense}} = \sum_{\boldsymbol{u}} w[\boldsymbol{u}] \left\| \Pi_{\boldsymbol{k}}\left(\mathbf{T}_j^{-1}\mathbf{T}_i \circ \Pi_{\boldsymbol{k}}^{-1}(\mathbf{D}_i[\boldsymbol{u}])\right) - \boldsymbol{u} - \mathbf{F}_{ij}[\boldsymbol{u}] \right\|^2, \qquad e_{\mathrm{depth}} = \sum_{\boldsymbol{u}} m[\boldsymbol{u}] \left\| \mathbf{D}_i[\boldsymbol{u}] - \mathbf{D}_i^{\mathrm{prior}}[\boldsymbol{u}] \right\|^2, \tag{17}$$

where $\mathbf{F}_{ij}$ is the flow from a learned network (with the current geometry as a prior) and w its confidence; e_{sparse} is the same reprojection for Shi–Tomasi corners tracked by Lucas–Kanade on the full-resolution image (cuVSLAM), implemented by bilinear splatting into the flow form to keep the Hessian structure; $\mathbf{D}^{\mathrm{prior}}$ comes from a monocular metric depth network (Metric3Dv2 in all reported experiments; UniDepthV2, UniK3D selectable) and is re-predicted when the intrinsics change. Solved by Gauss–Newton with a factorised sparse solver and COLAMD ordering. **Dynamic masking**: GroundingDINO boxes for user-specified classes prompt SAM at a fixed interval, propagated with XMem; the inverted mask multiplies the flow weights and removes sparse tracks. **Camera models**: a radial model $\boldsymbol{u} = (f q_{\boldsymbol{k}}(\theta)\cos\phi + W/2,\ f q_{\boldsymbol{k}}(\theta)\sin\phi + H/2)$ with principal point fixed at the centre; pinhole $q = \tan\theta$; wide-angle by the unified camera model $q = \tan\theta/(1 + \alpha\sqrt{\tan^2\theta + 1})$, $\boldsymbol{k} = [f, \alpha]$; rigs by $\mathbf{T}_v\mathbf{T}_i$; 360° as six cube-face pinholes.

11.3 Dense depth alignment (paper Eq. 6)

A video depth network gives affine-invariant $\mathbf{D}_i^{\mathrm{VDA}}$; the BA geometry gives a sparse $\mathbf{D}_i^{\mathrm{BA}}$; per frame $\alpha_i, \beta_i = \arg\min \sum \|\mathbf{M} \cdot (\alpha/\mathbf{D}^{\mathrm{VDA}} + \beta - 1/\mathbf{D}^{\mathrm{BA}})\|^2$, smoothed with momentum, $\mathbf{D}_i^{\mathrm{HD}} = 1/(\hat{\alpha}_i/\mathbf{D}_i^{\mathrm{VDA}} + \hat{\beta}_i)$; sparse coverage is infilled with PriorDA.

11.4 Reported results (inherited)

TUM-RGBD: ATE 3.6 cm (static), 1.5 cm (dynamic), focal error 1.8°/0.6°, ≈3 min per ≈950 frames on an RTX 5090. KITTI (first 1024 frames, cropped to 512×368): ATE 9.2 m, focal error 1.9°; RDS: ATE 5.0 m, focal 7.9°; 3–5 FPS at 640×480. Without ground truth they propose a *shuttle* metric (run the video forwards and reversed, compare) and the Sampson epipolar error. Ablation (OpenDV): dense only S-ATE $1.39 \cdot 10^{-2}$; all terms with masking $1.05 \cdot 10^{-2}$. Datasets: Dynpose-100K++ (99,501 videos, 15.7 M frames), Wild-SDG-1M (≈78 M frames), Web360 (≈2,000 videos); ≈96 M frames in total.

11.5 Comparison

aspect	our pipeline	ViPE
inputs	video + IMU + GNSS + orientation	video only
estimator	EKF forward + RTS backward over the session; 6 states	keyframe dense BA (Gauss–Newton) over poses, depth, intrinsics
metric scale	GNSS distance and speed; IMU	monocular metric-depth prior (αe_{depth})
time sync	explicit: PTS, coarse anchors, three-band gyro cross-correlation	not applicable
intrinsics	gyro regression, nominal FOV, plausibility gate; wrong on the ultra-wide	GeoCalib initialisation, refined in backend BA; unified model for wide lenses
camera motion	one mount per session (or an attitude track)	6-DoF per frame
vehicle	phone→vehicle frame, rear-axle reference, steering	none
road	ground plane, lanes, horizon	dense depth; no road model
dynamics	ignored	GroundingDINO + SAM + XMem masks
car interior	unhandled (S24 failure)	masked only if a class prompt matches
uncertainty	full covariance per instant; hold-out errors	BA residuals
self-checks	quality gates; prediction vs outcome	shuttle and Sampson metrics
compute	CPU; ≈0.4–0.75 s per rendered 1080p frame	GPU; 3–5 FPS

Our approach is strong where the vehicle and its sensors are known; ViPE where only pixels are. On our recordings ViPE's learned intrinsics prior plus BA would have found the ultra-wide focal length, and per-frame poses would have absorbed EIS motion without a hand-built track. It would not give a vehicle-referenced trajectory or steering, its scale depends on how a depth network generalises to a dashcam view, and nothing in it knows that a third of our frame is the inside of the car.

12 Plan: a fused geometric video engine

12.1 Target formulation

Replace the separate stages (sync, mount, EKF+RTS, calibration) by one sliding-window optimisation over keyframes i with states $\boldsymbol{x}_i$ (pose $\mathbf{T}_i \in \mathrm{SE}(3)$, velocity, IMU biases), the static parameters $\Theta = \{\boldsymbol{k}, \mathbf{R}_{cv}, \boldsymbol{t}_v, \delta t\}$ (intrinsics with a distortion term, camera-to-vehicle mount, sensor-to-video time offset) and low-resolution keyframe inverse depth $\mathbf{D}_i$:

$$
\begin{aligned}
E = &\sum_i \|r_{\text{IMU}}(\boldsymbol{x}_i, \boldsymbol{x}_{i+1})\|^2_{\Sigma_{\text{IMU}}} + \sum_g \rho\big(\|r_{\text{GNSS}}(\boldsymbol{x}, t_g)\|^2_{\Sigma_g}\big) \\
&+ \sum_{(i,j)} \sum_{\boldsymbol{u} \in \mathcal{M}_s} w_{ij}[\boldsymbol{u}] \left\| \Pi_{\boldsymbol{k}}\big(\mathbf{T}_{cj}^{-1} \mathbf{T}_{ci}\, \Pi_{\boldsymbol{k}}^{-1}(\boldsymbol{u}, \mathbf{D}_i[\boldsymbol{u}])\big) - \boldsymbol{u} - \mathbf{F}_{ij}[\boldsymbol{u}] \right\|^2 \\
&+ \lambda_{\text{road}} \sum_i \sum_{\boldsymbol{u} \in \mathcal{G}_i} \big(\boldsymbol{n}^\top \mathbf{X}_i(\boldsymbol{u}) + h\big)^2 + \lambda_{\text{lane}} \sum_i r_{\text{lane}}(\mathbf{T}_i, \Theta)^2 \\
&+ \alpha \sum_i \|\mathbf{D}_i - s_i\, \mathbf{D}_i^{\text{prior}}\|^2_m + \|\Theta - \Theta_0\|^2_{\Sigma_\Theta},
\end{aligned}
\tag{18}
$$

with IMU preintegration between keyframes, GNSS position/velocity/course factors with a robust loss ρ (the Huber down-weighting of §8), dense flow over the *static* mask $\mathcal{M}_s$ (dynamic objects *and* car interior removed), a road-plane term on segmented ground pixels $\mathcal{G}_i$, lane parallelism and width constancy, the depth prior with a free per-keyframe scale s_i (so it cannot set metric scale), and priors Θ_0 from cheap measurements (gyro–image focal, lane VP). Camera poses are $\mathbf{T}_{ci} = \mathbf{T}_i\, \mathbf{T}_{vc}(\Theta)$, so every image term constrains the vehicle trajectory and the mount at once. The output is the smoothed vehicle trajectory with its marginal covariances, exactly what §8 produces today, plus intrinsics, mount, time offset and depth.

12.2 What is adopted from ViPE, and how it is developed further

ViPE method	adopted as	developed further
dense flow term e_{dense}	image factor between keyframes	expressed through the *vehicle* pose and mount; static mask includes the car interior, learned per recording from temporal variance and zero flow
sparse tracks e_{sparse}	full-resolution corner tracks	dash and lane-marking ends as metric-scale anchors (the dash measurement that fixed fh on the S24)
intrinsics in backend BA	$\boldsymbol{k}$ as a variable	initialised from gyro rotation vs far-field shift (which found the ultra-wide); plausibility set from the phone's real lens list
unified camera model	distortion parameter α	selected per lens; checked against straight lane lines
depth prior αe_{depth}	regulariser	free scale per keyframe, so GNSS/IMU set scale; prior–geometry disagreement logged as a domain-shift monitor
semantic masking	GroundingDINO/SAM-style masks	plus interior mask; plus ground segmentation for the road term
keyframes, frontend/backend	sliding window	IMU and GNSS factors marginalised at the window edge (bounded memory for hour-long drives)
shuttle metric	forward/reversed consistency	combined with our prediction-vs-outcome check and GNSS hold-out as release gates

12.3 Work packages

WP	package	content	pass criterion
0	Robust front end	resolution-independent thresholds; automatic interior/static mask; unified camera model	S24 recording calibrates with no override; mask IoU ≥ 0.9
1	Intrinsics from rotation	focal from far-field shift vs gyro, with distortion; lens-aware gate	$\lvert f - f_{\text{cb}}\rvert < 2\,\%$ vs checkerboard, main and ultra-wide
2	Fused optimiser	IMU preintegration + GNSS + dense flow; Θ including time offset; replaces EKF+RTS and mount calibration	hold-out position RMS $\leq$ today; prediction-vs-outcome at 30 m $\leq$ 0.2 m median
3	Road and lane factors	ground-plane and lane terms; separates f from h with GNSS scale	h within 3 cm of tape; v_h within 3 px of lane-width estimate
4	Time-varying mount	spline rotation for EIS / loose holders, from car-fixed features and image–gyro residuals	EIS recording: path-vs-lane within $\pm 0.5^\circ$ in every 5 s bin
5	Depth prior as regulariser	scale-free prior; ablation	GNSS scale unchanged when the prior is dropped
6	Dynamics and long drives	masks for vehicles and people; marginalising window; loop closure	no degradation in dense traffic; constant memory at 1 h
7	Evaluation harness	per recording: trajectory hold-out, intrinsics vs checkerboard, mount vs tape, drawing error per time bin; plus KITTI/nuScenes	runs on every release; regressions fail CI

Order. WP0 and WP1 first (days; they make the S24 recording work without manual overrides) with WP7 in parallel; then WP2 with WP3 (weeks); WP4–6 after. Code and models are chosen independently of ViPE and their licences checked (ViPE is Apache-2.0 with non-commercial components).

13 Recording recommendations

- Record with the main (1×) camera at 1080p unless the ultra-wide is wanted; note the lens. Keep stabilisation off.

- Mount rigidly, with as little dashboard in view as possible.
- Measure camera height and lateral position once per mount; one checkerboard video per phone and lens.
- Start with ten seconds stationary (gyro bias, gravity reference).

14 Limitations

- The S24 corrected calibration is measured on one 60 s window and assumes a distortion-corrected ultra-wide image; it is not yet validated over the full 11 minutes.
- ViPE results are INHERITED from the paper; several AUDIT.md figures are INHERITED from the original project.